

Abominable Function Types

P0172R0 - 2015-11-10

Alisdair Meredith, ameredith1@bloomberg.net

Table of Contents

- [1 Introduction](#)
- [2 Abominable Function Types](#)
 - [2.1 Definition](#)
 - [2.2 Basic Issues](#)
 - [2.3 How are Abominable Function Types Used?](#)
 - [2.4 How are Abominable Function Types *Really* Used?](#)
 - [2.5 Why do Abominable Function Types Exist?](#)
- [3 Concerns](#)
- [4 Recommended Direction](#)
 - [Option 1: Remove the abominations](#)
 - [Option 2: Abominations are not Function](#)
 - [Option 3: Make Abominations More Regular](#)
 - [Option 4: Embrace in the Library](#)

1 Introduction

There is a dark corner of the type system that is little known other than to compiler writers, and authors of generic code who must document which types their templates can handle. This paper tries to shed some light in the dark corners, and explores whether the language can be cleaned up so that good template documentation does not drag obscure language-lawyer details into the domain of the regular user.

2 Abominable Function Types

2.1 Definition

For the purposes of this paper, an *abominable* function type is the type produced by writing a function type followed by a *cv-ref* qualifier.

Example:

```
using regular      = void();
using abominable = void() const volatile &&;
```

In the example above, `regular` names a familiar function type and should contain no surprises. `abominable` also names a function type, not a reference type, and despite appearances, is neither a `const` nor a `volatile` qualified type. There is no such thing as a *cv*-qualified function type in the type system, and the abominable function type is something else entirely.

Note that it is not possible to create a function that has an abominable type. Rather, the *cv-ref* qualifier applies to the implicit `*this` reference when calling a member function. However, abominable function types are specifically function types, and not member function types. This is evident from the lack of ability to specify the type of class the abominable function would be a member of, when declaring such a function type.

2.2 Basic Issues

The inability to create a function of such types goes deep into the language. Given it is not possible to create a function of such a type, the language further protects us by making it ill-formed (diagnostic required) to form a reference or a pointer to such a type, i.e., there are no abominable function reference types, and no abominable function pointer types. Take the following example:

```
using abominable = void() const volatile &&;
using const_type = abominable const;
using pointer    = abominable *;      // ill-formed
using reference  = abominable &;      // ill-formed
```

In this case, `abominable` names a function type, and as per the rules for function types, and `const_type` names the same type as

abominable as cv qualifiers are ignored for function types. However, unlike regular function types, the lines attempting to define pointer and reference are ill-formed.

2.3 How are Abominable Function Types Used?

Outside generic code, where the language syntax permits instantiating templates on any type, how would an abominable type be used in regular code?

The only examples I have of explicitly writing these types fall into the category of showing off knowledge of the corners of compilers, and winning obfuscated coding contests. I have not yet encountered the following idiom in real-world usage outside of such scenarios.

```
class rectangle {
public:
    using int_property = int() const;    // common signature for several methods

    int_property top;
    int_property left;
    int_property bottom;
    int_property right;
    int_property width;
    int_property height;

    // Remaining details elided
};
```

Here we declare a `rectangle` class with accessors for common properties of a rectangle, which would doubtlessly compute the value, rather than store directly, for some of these properties. This is a very common object oriented example so I will not labour the details. The key is that the class author wants to guarantee a regularity of declaration for all member access functions, so forms an abominable function type with the desired `const` qualified access, to ensure that all functions have identical signatures.

Note that this example applies to function types in general, not just abominable function types. However, most other examples of using function types do not apply to abominable function types, due to their inability to form references and pointers. While the above idiom with regular function types is highly unusual, even in tutorials on the esoteric, I am not aware of any such coverage that actually uses abominable function types when demonstrating the idiom, so even this usage is yet to be proved as more than theoretical.

2.4 How are Abominable Function Types *Really* Used?

I run into two related uses of abominable function types in production code bases. The most common is writing test drivers for templates, where we want to be sure that the template robustly handles all the types that it claims to handle. Testing the standard library type traits is an important use case, as most type traits claim to handle any (complete) type.

The second use case is implementing special workarounds in templates that claim to accept any type, or any function type. For example, I would really like my implementation of `std::is_function_v` to be this simple:

```
template <typename TYPE>
constexpr bool is_function_v = false;

template <typename RESULT, typename ...ARGS>
constexpr bool is_function_v<RESULT(ARGS...)> = true;
```

Commonly, I will forget about functions with a C-style ellipsis, which require one further specialization:

```
template <typename TYPE>
constexpr bool is_function_v = false;

template <typename RESULT, typename ...ARGS>
constexpr bool is_function_v<RESULT(ARGS...)> = true;

template <typename RESULT, typename ...ARGS>
constexpr bool is_function_v<RESULT(ARGS.....)> = true;
```

However, abominable functions types are still function types, so we need to produce the result `true` for any abominable function type as well:

```
template <typename TYPE>
constexpr bool is_function_v = false;

template <typename RESULT, typename ...ARGS>
constexpr bool is_function_v<RESULT(ARGS...)> = true;
```

```

template <typename RESULT, typename ...ARGS>
constexpr bool is_function_v<RESULT(ARGS.....)> = true;

template <typename RESULT, typename ...ARGS>
constexpr bool is_function_v<RESULT(ARGS...) &> = true;

template <typename RESULT, typename ...ARGS>
constexpr bool is_function_v<RESULT(ARGS.....) &> = true;

template <typename RESULT, typename ...ARGS>
constexpr bool is_function_v<RESULT(ARGS...) &&> = true;

template <typename RESULT, typename ...ARGS>
constexpr bool is_function_v<RESULT(ARGS.....) &&> = true;

template <typename RESULT, typename ...ARGS>
constexpr bool is_function_v<RESULT(ARGS...) const> = true;

template <typename RESULT, typename ...ARGS>
constexpr bool is_function_v<RESULT(ARGS.....) const> = true;

template <typename RESULT, typename ...ARGS>
constexpr bool is_function_v<RESULT(ARGS...) const &> = true;

template <typename RESULT, typename ...ARGS>
constexpr bool is_function_v<RESULT(ARGS.....) const &> = true;

template <typename RESULT, typename ...ARGS>
constexpr bool is_function_v<RESULT(ARGS...) const &&> = true;

template <typename RESULT, typename ...ARGS>
constexpr bool is_function_v<RESULT(ARGS.....) const &&> = true;

template <typename RESULT, typename ...ARGS>
constexpr bool is_function_v<RESULT(ARGS...) volatile> = true;

template <typename RESULT, typename ...ARGS>
constexpr bool is_function_v<RESULT(ARGS.....) volatile> = true;

template <typename RESULT, typename ...ARGS>
constexpr bool is_function_v<RESULT(ARGS...) volatile &> = true;

template <typename RESULT, typename ...ARGS>
constexpr bool is_function_v<RESULT(ARGS.....) volatile &> = true;

template <typename RESULT, typename ...ARGS>
constexpr bool is_function_v<RESULT(ARGS...) volatile &&> = true;

template <typename RESULT, typename ...ARGS>
constexpr bool is_function_v<RESULT(ARGS.....) volatile &&> = true;

template <typename RESULT, typename ...ARGS>
constexpr bool is_function_v<RESULT(ARGS...) const volatile> = true;

template <typename RESULT, typename ...ARGS>
constexpr bool is_function_v<RESULT(ARGS.....) const volatile> = true;

template <typename RESULT, typename ...ARGS>
constexpr bool is_function_v<RESULT(ARGS...) const volatile &> = true;

template <typename RESULT, typename ...ARGS>
constexpr bool is_function_v<RESULT(ARGS.....) const volatile &> = true;

template <typename RESULT, typename ...ARGS>
constexpr bool is_function_v<RESULT(ARGS...) const volatile &&> = true;

template <typename RESULT, typename ...ARGS>
constexpr bool is_function_v<RESULT(ARGS.....) const volatile &&> = true;

```

Rather than a simple primary template and two partial specializations, there are now twenty-four partial specializations to write and test. Note that this paper was written before [P0012R1](#) was adopted, making exception specifications part of the type system, potentially doubling the number of partial specializations again.

2.5 Why do Abominable Function Types Exist?

If abominable function types have such peculiar properties, and no clear motivating use case, why are they in the language? The answer is that they were not designed to be this way, but fall out of the type system in an important, but relatively obscure, way.

As far as I can tell, the strange inconsistencies are the Core Working Groups best effort to steer us out of trouble should we ever encounter such a thing in the wild, and before template metaprogramming with generated types became a common idiom, and the `std::function` proposal for Library TR1 put function types directly in the spotlight.

```
struct host {
    int function() const; // { return 42; }
};

template <typename TYPE>
constexpr bool test(TYPE host::*) {
    return is_same_v<TYPE, int() const>;
}

constexpr auto member = &host::function;

static_assert(test(member), "This is why abominations exist");
```

In this example, we want to know the type that the pointer-to-member `host::*` refers to, for the member `function`. The `cv`-qualifier on the member is vital to `const` correct type safety in the language. Clearly, the compiler needs a distinct type to represent this, and users are going to need a way to spell that type if they are to interact using it.

3 Concerns

The main problem caused by abominable function types occurs in generic code. Templates must document the constraints on the types that their type parameters can be instantiated with, or can be assumed to accept any type. For example, there should be no constraints on the type parameters to `is_same`, as the behavior of the type is not relevant to the result. For a trait like `is_constructible`, you need to consider whether it is reasonable to support any type, including incomplete types (including `void`) or whether to limit the types that they support. Note that it is often reasonable to have a trait return `false` for a meaningless question, such as whether `void` types are copy constructible, rather than exclude them from the supported set of types.

In many cases, the simplest option for handling abominable function types in a trait (or other metaprogram) is simply to state that they are not supported. The main problem here is a lack of vocabulary, and every library author (who takes documenting their contracts seriously) will invent their own terminology to describe these things, and often be left with the burden of educating their users that these awkward types exist and what they are, only to explain that they are not supported.

Once we decide to support all types in our trait (or other template), if the implementation of the metafunction would involve forming a reference or a pointer, we must implement an extra layer of dispatch to detect abominable function types to handle them specially. This was the approach adopted by the standard library, although it still leaves the burden of documenting the result of traits when passed types that require such special dispatch.

The final concern is that the mere existence of these types greatly complicates the testing of templates, as rather than using a small variety of function types to test a trait against (functions taking 0, 1, or more arguments, and maybe with a C-style ellipsis and an awkward return type), we multiply that set by 12 to check that there is no unguarded path through the template forming a reference or pointer to an arbitrary dependant type, include the type parameters themselves.

4 Recommended Direction

There are a number of simple to not-so-simple approaches that could be taken in the standard to simplify these problems. Four possible directions are described below, with a weak recommendation for both of options 2 and 4.

Option 1: Remove the abominations

From the generic library author's perspective, this would be the ideal solution. If the problem is removed at source, then there is no need to worry about it. However, the example in [2.5](#) is real, and would need some new, distinctly novel, way to express the deduced parameter type. The problems of these types don't go away entirely, but are more easily expressed.

This is clearly the riskiest approach, leading to some real code breakage if the new model is a complete replacement for the old, or a period of even more confusion of both facilities live side-by-side during a period of deprecation. The author no longer expects this option to be a success.

Option 2: Abominations are not Function

Very similar to [Option 1](#), but rather than invent a new syntax we retain the existing form, but state that *function-like* types that appear syntactically like a function with a `cv`-qualifier are, in fact, not function types at all.

This option resolves the worst parts of the issue, which is that in generic code I do not have the simple guarantee that I can form

a pointer to an arbitrary function type, nor form a reference to such a type. This simplifies the burden on documenters of generic type manipulation libraries, and makes life simpler for their customers as disparate libraries each find their own distinct way to describe abominations that might not be supported.

Note that making this change in Core would have implications on the existing library documentation for traits like `is_function`. If abominations cease to be functions, then the majority of the existing library documentation would be valid, but the contracts would have changed meaning, generally leading to simpler implementations, but returning different results in the kind of corner case seen throughout this paper.

Option 3: Make Abominations More Regular

The abominations would cease to be abominable if they were more regular. The two simplest changes to improve regularity would be to allow pointers and references to such types, even though we know of no actual functions that could be bound to such references, and the only (known) valid pointer values would be `nullptr`.

This change would solve the majority of template metaprograms that fail with a hard error, as these assumptions are typically made in non-SFAINE contexts.

This option provides no help for the implementation for `is_function_v`, it remains the library implementer's responsibility to be aware of all the dark corners of the type system, and handle them appropriately.

Option 4: Minimal Library Cleanup

This is [Option 2](#), but applied entirely in the library. The main drawback compared to [Option 2](#) is that the core and library clauses either use different terms to describe the same things, or use the same terms to mean different things.

The process to clean up the library started with [LWG #2196](#) which was resolved for C++14 at the Bristol meeting. However, there is no `is_referenceable` trait (nor a corresponding `is_pointable`) despite the term being used as part of the precondition to a number of traits in the library, potentially making it tricky for users to correctly forward types in generic some generic code.

Other concerns include more clearly defining the behavior of `add_lvalue_reference_t<int() const>`, which is currently an alias to an invalid type (and so, presumably, should be a hard error in user code) unlike `add_lvalue_reference_t<void>`, which is an alias back to `void`. The precondition should be more clearly spelled out, or going with [Option 2](#) above, would immediately become well defined behavior to return the same type, as per `void`. (If it is intended that it should be an error to invoke `add_lvalue_reference_t` on such types, it should be clearly stated as a precondition, as is done for other standard type traits, or at the very least acknowledged in a note as likely to fail.)

Option 4: Embrace in the Library

This option extends, [Option 4](#), and could equally be an extension to each of the preceding options. The standard library, in particular the type traits component, should be extended to properly recognise and manipulate this vocabulary of types.

In addition to adding `is_referenceable` and `is_pointable`, and fixing inconsistencies with `add_const` and `add_pointer`, add the following list of traits for manipulating the *cv-ref* qualifier on function types:

- `has_member_qualifiers`
- `is_const_member`
- `is_volatile_member`
- `is_cv_member`
- `is_lvalue_reference_member`
- `is_rvalue_reference_member`
- `add_member_const`
- `add_member_volatile`
- `add_member_cv`
- `add_member_lvalue_reference`
- `add_member_rvalue_reference`
- `remove_member_const`
- `remove_member_volatile`
- `remove_member_cv`
- `remove_member_lvalue_reference`
- `remove_member_rvalue_reference`
- `remove_all_member_qualifiers`

Note that the traits adding and removing *cv*-qualifiers are peers of the traits adding and removing reference qualifiers, unlike the regular traits for adding and removing *cv*-qualifiers from types, which have no effect on reference types.

Some obvious issues with this proposal are deferred until LEWG feedback recommends taking this direction. For example, it is not clear whether the following should be supported, or what it should mean:

```
using result = add_lvalue_member_reference_t<void() &&>;
```

Similarly, how should these metafunctions be interpreted when passed a non-function type. Should they return the type unmodified, such as `add_reference_t<void>`, or should we consider the general case of how these would be affected through pointer-to-member types, where adding or removing `cv-` qualification from the dereferenced `this` pointer would be similar to adding or removing the `cv-` qualifier on the type of the data member?

These missing traits might also be just one aspect of a more complete type manipulation facility for function types, which would be useful for compile-time reflection.